

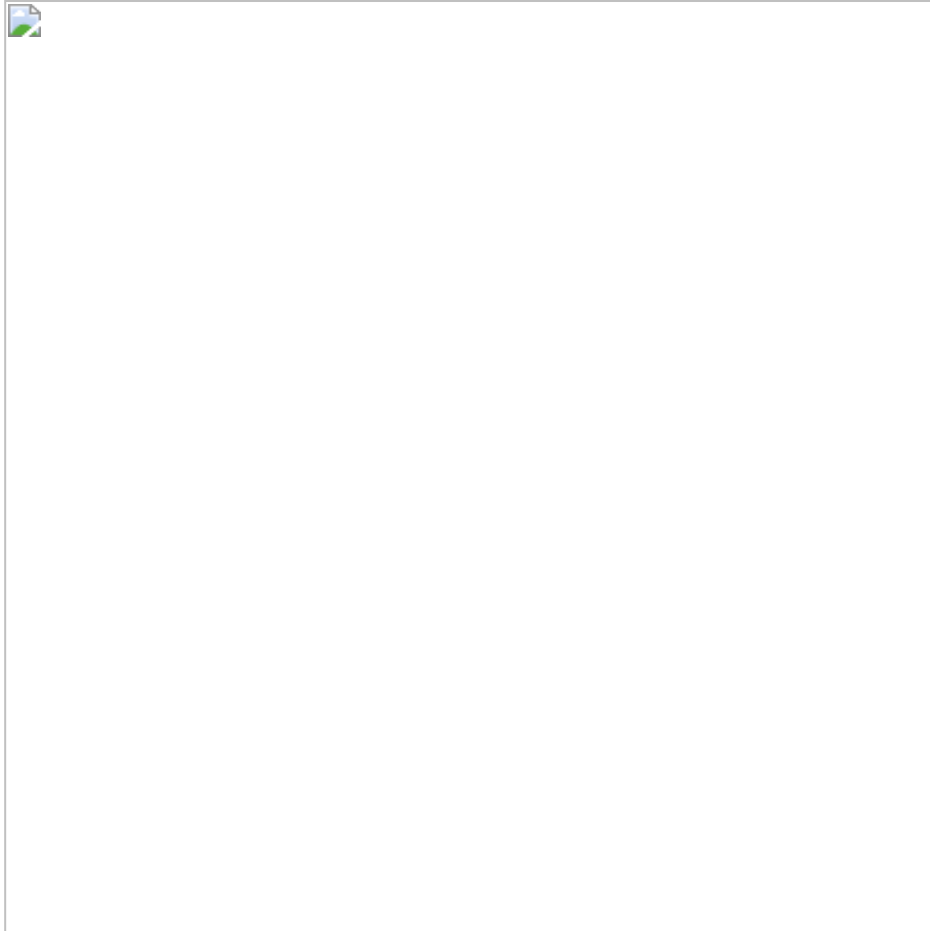
slides/slides/13.SC4_Conteneurs-et-orchestration/01.- Docker-intro/slides.md at master · O-clock-Cyber/slides

github.com/O-clock-Cyber/slides/blob/master/slides/13.SC4_Conteneurs-et-orchestration/01.-Docker-intro/slides.md



Docker

Conteneurisation



Docker est un logiciel permettant d'exécuter des applications dans des **conteneurs**.

Un conteneur ?



Ce truc-là ?

--

Pas tout à fait, nous on parle de **conteneurs logiciels**.

Mais le terme "conteneur" est bien inspiré des conteneurs maritimes, d'ailleurs le logo de Docker représente une "baleine porte-conteneur" !

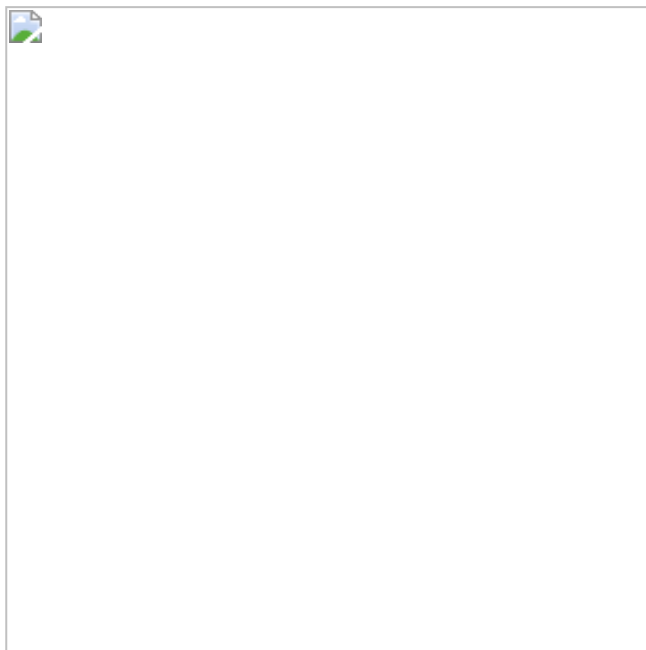


Ça sert à quoi, Docker ?

On doit souvent gérer des **problèmes de versions** en développement d'applications.

Par exemple : si le développeur a créé un site web avec PHP version 8, et qu'un administrateur doit le déployer sur un serveur avec seulement PHP version 7 d'installé, le site ne fonctionnera pas !

--



Et bien c'est justement l'idée derrière Docker :

Si ça fonctionne sur l'environnement de dev, **on livre l'environnement de dev en prod !**

--

Docker permet, entre autres :

- de plus facilement déployer une application en production
- de pouvoir **mettre en place rapidement un environnement de développement identique à l'environnement de production**
- de supporter les montées en charge

--

| Mais tout ça, c'est utile que pour les devs ?

Non ! Ce sont les admins sys qui s'occupent de la mise en production des applications en général.

Et il y a un autre intérêt pour nous : pouvoir installer facilement plein d'applications sur un même serveur très facilement !

Ça fonctionne comment ?

Il y a deux termes à connaître pour bien comprendre comment Docker fonctionne :

- les **conteneurs Docker**
- les **images Docker**

Conteneur Docker

Un conteneur est **une "enveloppe" virtuelle qui permet de distribuer une application avec tous les éléments dont elle a besoin pour fonctionner** : code source / binaire de l'application, environnement d'exécution, bibliothèques, outils et fichiers divers.



--

Prenons un premier exemple : **le frontend d'une application web** (ce qui est chargé dans le navigateur du visiteur).

Un conteneur permettant de distribuer un frontend avec tout ce dont il a besoin pour fonctionner peut par exemple être composé de :

- **application** : code source HTML/CSS/JS du frontend
- **environnement d'exécution** : serveur web Apache
- **bibliothèques** : on pourrait avoir une bibliothèque pour faire des animations
- **outils et fichiers divers** : on pourrait avoir ici NPM, un gestionnaire de dépendances

--

Deuxième exemple : **le backend d'une application web** (le code coté serveur, qui génère ce qui sera envoyé au navigateur).

Un conteneur permettant de distribuer un backend avec tout ce dont il a besoin pour fonctionner peut être composé de :

- **application** : code source PHP (ou Python, JS, Java, C#, etc.) du backend
- **environnement d'exécution** : serveur web Apache pour PHP, NodeJS pour JS, etc.
- **bibliothèques** : dossier `vendor` pour PHP, dossier `node_modules` pour JS, etc.
- **outils et fichiers divers** : Composer pour PHP, NPM et PM2 pour JS, etc.

Image Docker

En POO, on dit **qu'un objet est l'instance d'une classe**.

Avec Docker : **un conteneur Docker est l'instance d'une image**.

--

On peut créer **un nombre infini de conteneurs à partir d'une même image Docker** !



Mais concrètement, c'est quoi une image ?

Une image Docker est un fichier **immuable** (qu'on ne peut pas modifier) qui est une **capture instantanée du système de fichiers** d'un conteneur.

En quelques sortes, c'est comme si on prenait une "photo" du contenu du disque dur d'un serveur après avoir tout installé (l'application web + tout ce dont elle a besoin pour fonctionner).

Virtualisation vs. Conteneurisation

--

On peut faire le parallèle avec la virtualisation :

Le disque dur des **machines virtuelles** (Virtual Machine / VM en anglais) que nous utilisons depuis le début de la formation sur VirtualBox est un fichier avec l'extension **.vdi**.

--



On peut créer plusieurs **instances** (plusieurs VMs = équivalent à un conteneur) à partir d'un même disque dur virtuel (équivalent à une image Docker).

--

Différences entre virtualisation et conteneurisation

Une machine virtuelle embarque un **système d'exploitation complet**, ce n'est pas le cas avec un conteneur Docker, qui partage certains composants avec l'hôte sur lequel il est lancé (notamment **le noyau**).

En conséquence, une machine virtuelle peut être très lourde (de plusieurs Gigaoctets à plusieurs dizaines de Go !), là où **un conteneur Docker sera plus léger** (quelques centaines de Mo en général).

--



Images immuables

Les images Docker, en quelques sortes les "disques durs virtuels" de nos conteneurs, sont **immuables**.

Ça veut dire qu'**on ne peut pas les modifier**. *Mais comment on fait du coup pour installer nos logiciels et y placer notre code ?* 🤔

--

Dockerfile

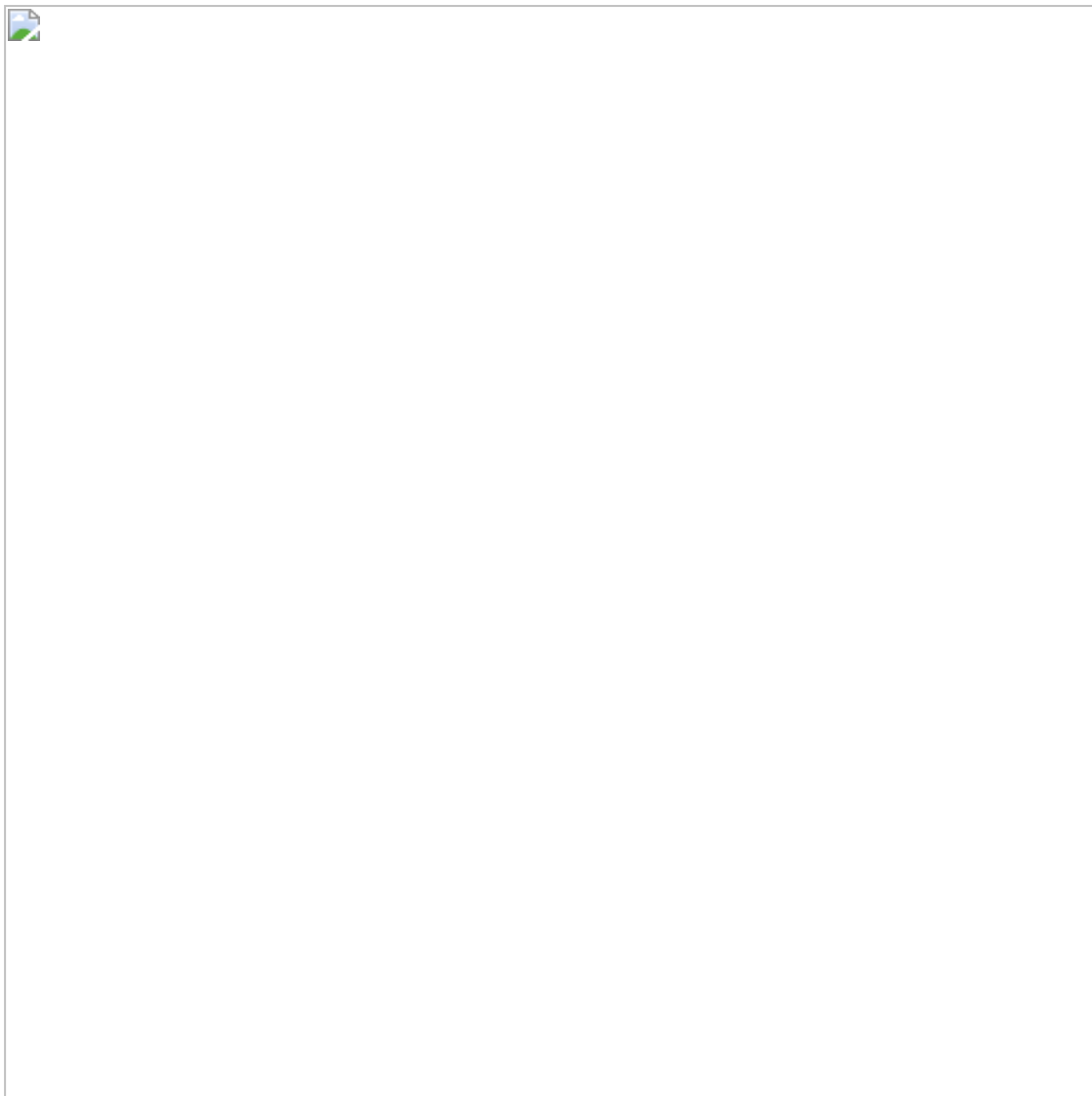
Pour créer **notre propre image Docker**, on va partir d'une **image existante**.

Grâce à un fichier de configuration spécifique à Docker, le **Dockerfile**, on va pouvoir indiquer à Docker ce qu'on veut **ajouter** sur l'image existante.

Nous découvrirons la syntaxe et les commandes à utiliser pour rédiger et utiliser un Dockerfile demain.

--

Exemple pour créer une image Docker de notre frontend :



Le **Dockerfile** permet de copier des fichiers dans une image, lancer des commandes, installer des logiciels ou dépendances, etc.

L'image d'origine, embarquant le logiciel Apache2, a elle aussi été construite en utilisant un Dockerfile.

--

Pour utiliser Docker avec nos projets, nous n'avons **pas forcément besoin de créer nos propres images Docker avec un Dockerfile !**

On peut se contenter d'utiliser des images existantes, comme par exemple celle d'Apache2.
Mais on les récupère où, ces images ? 🤔

DockerHub

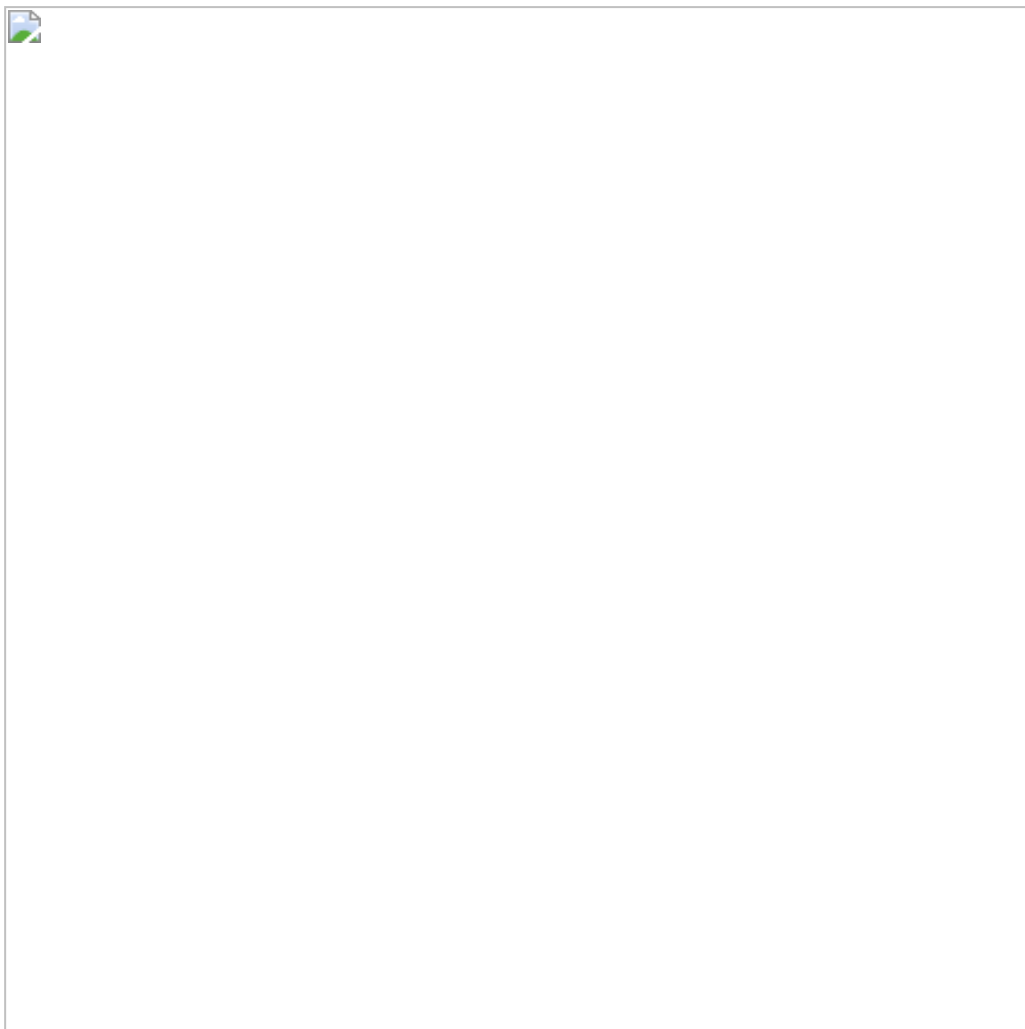
Comme pour Git & Github, avec Docker on a **DockerHub** !

<https://hub.docker.com/>

--

DockerHub est un site web sur lequel on peut **héberger nos images Docker dans des dépôts** (repository).

Comme sur Github avec nos dépôts Git, on pourra choisir si nos dépôts d'images Docker sont **publics ou privés**. Avec l'offre gratuite, nous sommes limités à un seul dépôt privé.



--

Dans les nombreux dépôts publics, on retrouve des images Docker prêtes à l'emploi qui vont nous être très utiles comme base pour **construire nos propres images** ou mettre en place nos **environnements de développement et de production**.



Explorez les suggestions !

Docker Compose

On va donc devoir créer une image contenant le frontend, le backend, et la base de données d'une application ? 🤔

Non ! On essaye en règle générale avec Docker de faire les choses de façon **modulaire**.

--

Applications multi-conteneurs

Une application web va plutôt être **divisée en trois conteneurs** :

- un conteneur pour le frontend
- un conteneur pour le backend
- un conteneur pour la base de données

Ainsi, on peut facilement **mettre à jour chaque conteneur indépendamment**. On peut aussi, sur des applications plus sollicitées, démarrer plusieurs conteneurs pour le front ou le back, afin de supporter **un plus grand nombre de requêtes**.

--

3 conteneurs, 3 images

On l'a dit, **un conteneur est l'instance d'une image Docker**.

Il va donc nous falloir **trois images Docker, une pour chaque conteneur de notre application**.

Pour faire fonctionner une **application multi-conteneurs**, on utilisera **Docker Compose**, un "plugin" de Docker. On utilisera un fichier au format **YAML** (une alternative au **JSON**) pour configurer Docker Compose.

Docker : pratique

Avant tout, il faut qu'on **installe Docker** !

Pour l'instant, on va l'installer **sur une VM Linux**, et découvrir les commandes de base de Docker !